the 3-D space along lines parallel to the cartesian axes and passing through the point.

In general, for a k-dimensional space, we will need 2^k children for each node in the tree, because each point would partition the hyperspace in 2^k parts.

For real datasets, with the advent of big data, we will have to deal with high-dimensional spaces, meaning that k might easily be in the order of 10 to 30, or even 100. It's not unusual for datasets to have hundreds of features, and millions of points, and there are use cases where we need to perform nearest neighbor search on these datasets to make sense of them (clustering is one, as we will see in chapter 12).

Even with a smaller number of features, in the order of 10, each node would already have around a thousand children: as we have seen in chapter 2, talking about d-way heaps, when the branching factor of a tree grows too much, the tree flattens and becomes closer to a list.

But with 100-dimensional datasets, the number of children per node would be closer to 10^{33}, a number so large that it becomes challenging to store even a single node of such a tree.

We certainly need to do better than that. But how?

As we will see in the next couple of chapters, computer scientists found a few different ways to cope with these issues. In next chapter, in particular, we introduce k-d trees,